while the inverse cdf is given in terms of the inverse of $I_x(\alpha,\beta)$ on its subscript argument by

$$x(P) = I_p^{-1}(\alpha,\beta) \tag{6.14.55}$$

incgammabeta.h

```
struct Betadist : Beta {
Beta distribution, derived from the beta function Beta.
    Doub alph, bet, fac;
    Betadist(Doub aalph, Doub bbet) : alph(aalph), bet(bbet) {
    Constructor. Initialize with α and β.
        if (alph <= 0. || bet <= 0.) throw("bad alph,bet in Betadist");
        fac = gammln(alph+bet)-gammln(alph)-gammln(bet);
    }
    Doub p(Doub x) {
    Return probability density function.
        if (x <= 0. || x >= 1.) throw("bad x in Betadist");
        return exp((alph-1.)*log(x)+(bet-1.)*log(1.-x)+fac);
    }
    Doub cdf(Doub x) {
    Return cumulative distribution function.
        if (x < 0. || x > 1.) throw("bad x in Betadist");
        return betai(alph,bet,x);
    }
    Doub invcdf(Doub p) {
    Return inverse cumulative distribution function.
        if (p < 0. || p > 1.) throw("bad p in Betadist");
        return invbetai(p,alph,bet);
    }
};
```

6.14.12 Kolmogorov-Smirnov Distribution

The *Kolmogorov-Smirnov* or *KS* distribution, defined for positive z, is key to an important statistical test that is discussed in §14.3. Its probability density function does not directly enter into the test and is virtually never even written down. What one typically needs to compute is the cdf, denoted $P_{KS}(z)$, or its complement, $Q_{KS}(z) \equiv 1 - P_{KS}(z)$.

The cdf $P_{KS}(z)$ is defined by the series

$$P_{KS}(z) = 1 - 2\sum_{j=1}^{\infty}(-1)^{j-1}\exp(-2j^2z^2) \tag{6.14.56}$$

or by the equivalent series (nonobviously so!)

$$P_{KS}(z) = \frac{\sqrt{2\pi}}{z}\sum_{j=1}^{\infty}\exp\left(-\frac{(2j-1)^2\pi^2}{8z^2}\right) \tag{6.14.57}$$

Limiting values are what you'd expect for cdf's named "P" and "Q":

$$\begin{aligned} P_{KS}(0) &= 0 \qquad & P_{KS}(\infty) &= 1 \\ Q_{KS}(0) &= 1 \qquad & Q_{KS}(\infty) &= 0 \end{aligned} \tag{6.14.58}$$

Both of the series (6.14.56) and (6.14.57) are convergent for all $z > 0$. Moreover, for any z, one or the other series converges extremely rapidly, requiring no more than three terms to get to IEEE double precision fractional accuracy. A good place to switch from one series to the other is at $z \approx 1.18$. This renders the KS functions computable by a single exponential and a small number of arithmetic operations (see code below).

Getting the inverse functions $P_{KS}^{-1}(P)$ and $Q_{KS}^{-1}(Q)$, which return a value of z from a P or Q value, is a little trickier. For $Q \lesssim 0.3$ (that is, $P \gtrsim 0.7$), an iteration based on (6.14.56) works nicely:

$$\begin{aligned} x_0 &\equiv 0 \\ x_{i+1} &= \tfrac{1}{2}Q + x_i^4 - x_i^9 + x_i^{16} - x_i^{25} + \cdots \\ z(Q) &= \sqrt{-\tfrac{1}{2}\log(x_\infty)} \end{aligned} \tag{6.14.59}$$

For $x \lesssim 0.06$ you only need the first two powers of x_i.

For larger values of Q, that is, $P \lesssim 0.7$, the number of powers of x required quickly becomes excessive. A useful approach is to write (6.14.57) as

$$\begin{aligned} y\log(y) &= -\frac{\pi P^2}{8}\left(1 + y^4 + y^{12} + \cdots + y^{2j(j-1)} + \cdots\right)^{-1} \\ z(P) &= \frac{\pi/2}{\sqrt{-\log(y)}} \end{aligned} \tag{6.14.60}$$

If we can get a good enough initial guess for y, we can solve the first equation in (6.14.60) by a variant of Halley's method: Use values of y from the *previous* iteration on the right-hand side of (6.14.60), and use Halley only for the $y\log(y)$ piece, so that the first and second derivatives are analytically simple functions.

A good initial guess is obtained by using the inverse function to $y\log(y)$ (the function `invxlogx` in §6.11) with the argument $-\pi P^2/8$. The number of iterations within the `invxlogx` function and the Halley loop is never more than half a dozen in each, often less. Code for the KS functions and their inverses follows.

ksdist.h

```
struct KSdist {
Kolmogorov-Smirnov cumulative distribution functions and their inverses.
    Doub pks(Doub z) {
    Return cumulative distribution function.
        if (z < 0.) throw("bad z in KSdist");
        if (z == 0.) return 0.;
        if (z < 1.18) {
            Doub y = exp(-1.23370055013616983/SQR(z));
            return 2.25675833419102515*sqrt(-log(y))
                *(y + pow(y,9) + pow(y,25) + pow(y,49));
        } else {
            Doub x = exp(-2.*SQR(z));
            return 1. - 2.*(x - pow(x,4) + pow(x,9));
        }
    }
    Doub qks(Doub z) {
    Return complementary cumulative distribution function.
        if (z < 0.) throw("bad z in KSdist");
        if (z == 0.) return 1.;
        if (z < 1.18) return 1.-pks(z);
        Doub x = exp(-2.*SQR(z));
        return 2.*(x - pow(x,4) + pow(x,9));
    }
    Doub invqks(Doub q) {
```

Return inverse of the complementary cumulative distribution function.

```
    Doub y,logy,yp,x,xp,f,ff,u,t;
    if (q <= 0. || q > 1.) throw("bad q in KSdist");
    if (q == 1.) return 0.;
    if (q > 0.3) {
        f = -0.392699081698724155*SQR(1.-q);
        y = invxlogx(f);                         Initial guess.
        do {
            yp = y;
            logy = log(y);
            ff = f/SQR(1.+ pow(y,4)+ pow(y,12));
            u = (y*logy-ff)/(1.+logy);          Newton's method correction.
            y = y - (t=u/MAX(0.5,1.-0.5*u/(y*(1.+logy))));  Halley.
        } while (abs(t/y)>1.e-15);
        return 1.57079632679489662/sqrt(-log(y));
    } else {
        x = 0.03;
        do {                                     Iteration (6.14.59).
            xp = x;
            x = 0.5*q+pow(x,4)-pow(x,9);
            if (x > 0.06) x += pow(x,16)-pow(x,25);
        } while (abs((xp-x)/x)>1.e-15);
        return sqrt(-0.5*log(x));
    }
}
Doub invpks(Doub p) {return invqks(1.-p);}
```

Return inverse of the cumulative distribution function.

```
};
```

6.14.13 Poisson Distribution

The eponymous *Poisson distribution* was derived by Poisson in 1837. It applies to a process where discrete, uncorrelated events occur at some mean rate per unit time. If, for a given period, λ is the mean expected number of events, then the probability distribution of seeing exactly k events, $k \geq 0$, can be written as

$$\begin{aligned} k &\sim \text{Poisson}(\lambda), & \lambda > 0 \\ p(k) &= \frac{1}{k!}\lambda^k e^{-\lambda}, & k = 0, 1, \dots \end{aligned} \tag{6.14.61}$$

Evidently $\sum_k p(k) = 1$, since the k-dependent factors in (6.14.61) are just the series expansion of e^λ.

The mean and variance of Poisson(λ) are both λ. There is a single mode at $k = \lfloor \lambda \rfloor$, that is, at λ rounded down to an integer.

The Poisson distribution's cdf is an incomplete gamma function $Q(a, x)$,

$$P_\lambda(< k) = Q(k, \lambda) \tag{6.14.62}$$

Since k is discrete, $P_\lambda(< k)$ is of course different from $P_\lambda(\leq k)$, the latter being given by

$$P_\lambda(\leq k) = Q(k + 1, \lambda) \tag{6.14.63}$$

Some particular values are

$$P_\lambda(< 0) = 0 \qquad P_\lambda(< 1) = e^{-\lambda} \qquad P_\lambda(< \infty) = 1 \tag{6.14.64}$$